ReFiles: A Capability-Oriented Architecture for Responsive File Browsing Across Heterogeneous Storage Providers

Tomoyuki Terashita

Files Community

onein528@gmail.com

Abstract

A modern file manager must present local files, virtual Shell objects, archives, and remote storage through a coherent interface while preserving responsiveness under heterogeneous latency and threading constraints. This thesis presents ReFiles, a C# and WinUI 3 file manager organized around a UI-independent Core, optional item capabilities, progressive browse publication, and explicit resource ownership. Providers contribute semantic behaviors—including properties, thumbnails, previews, streams, operations, and change observation—through lazy factories, deterministic combiners, and wrappers instead of provider checks in generic code. Browse generations combine cooperative cancellation with stale-result rejection; bounded publication and viewport-driven enrichment separate first useful content from eventual metadata completion. Windows Shell, archive, and FTP/FTPS implementations demonstrate the model across affine COM objects, shared archive streams, and remote sessions. A five-run synthetic Core evaluation at 100, 1,000, 10,000, and 44,000 items observed median first-batch times below 0.16 ms and notification counts of 2, 4, 12, and 46, respectively. These measurements are not presented as UI or real-storage benchmarks; instead, they validate the bounded shape of the generic pipeline. The result is an architecture in which optional behavior, progressive presentation, and lifetime rules form one consistent boundary for extending a desktop file manager.

1 Introduction

1.1 The File Manager as a Systems Problem

A graphical file manager appears simple because its most visible operations are familiar: list a directory, open an item, copy or move it, and show a preview. Underneath that interface, however, a modern file manager coordinates several systems with different performance characteristics, failure modes, and ownership rules. Local files may be exposed through direct filesystem APIs or through the Windows Shell namespace. Archives behave like folders while requiring a decoding backend and, in some cases, credentials. Remote providers such as FTP add latency, partial failure, and session lifetimes. Property handlers, thumbnail providers, preview handlers, and change notifications introduce further platform-specific behavior. The user nevertheless expects all of these sources to appear within one coherent application.

The difficulty is not only functional coverage. A file manager is an interactive system, and its architecture is visible through latency. A directory may eventually enumerate correctly while still feeling unresponsive if the first useful row is delayed, if metadata work monopolizes the UI thread, or if thousands of fine-grained notifications overwhelm presentation. Conversely, aggressively parallel work may return after navigation has already changed, allowing stale properties or thumbnails to appear in a new view. Correctness therefore includes both the final result and the timing and lifetime boundaries through which that result travels.

ReFiles is an open-source desktop file manager implemented in C# and .NET with a WinUI 3 presentation layer [1]. It is developed as an alternative exploration of the architecture behind a Windows file manager rather than as a thin visual wrapper around one storage API. Its design separates UI-independent application and storage state from presentation, represents provider-specific optional behavior through capabilities, publishes browse results progressively, and isolates platform work behind explicit scheduling and ownership boundaries.

This thesis uses ReFiles as a concrete systems-engineering case study. The objective is not to claim that the individual ideas of capability discovery, asynchronous enumeration, or layered architecture are new in isolation.

Instead, it examines how these ideas can be combined into a practical file manager whose generic browsing code works across heterogeneous providers while remaining responsive and explicit about resource ownership.

1.2 Architectural Challenges

Three challenges shape the design: heterogeneous provider behavior, progressive presentation at scale, and the lifetime of asynchronous and platform-bound resources.

1.2.1 Heterogeneous storage providers

Storage providers share a small semantic core. They expose items with identities and names, resolve references, and usually offer some form of hierarchy or enumeration. Their optional behavior differs substantially. A local filesystem item may support efficient change notifications and Windows Shell properties. An archive entry may provide a readable stream but no native rename operation. An FTP item may expose remote metadata while lacking a thumbnail implementation. A virtual Shell item may support preview through a registered handler even when it does not map cleanly to a filesystem path.

A single universal interface tends to make these differences implicit. Optional members either throw when a provider does not implement them or require capability flags that must be kept consistent with the methods they guard. Provider type checks create a different problem: generic browsing and presentation code gradually learns about every backend, increasing coupling and making each new provider a cross-cutting change.

ReFiles instead treats optional behavior as a set of capabilities attached to an item. Generic consumers request the semantic behavior they need, such as property retrieval, thumbnail production, preview, stream access, or change observation. Absence is a valid result. A capability may be supplied directly by a core model, created by a provider-specific factory, composed from multiple candidates, or wrapped with cross-cutting behavior such as caching. This approach preserves a small required storage contract while allowing provider differences to remain behind UI-independent interfaces.

1.2.2 Latency and progressive feedback

Directory browsing is a pipeline rather than one operation. A location is resolved, items are enumerated, storage models are projected into browse state, changes are published to presentation, UI models are created or updated, and visual containers are realized. Properties and thumbnails enrich those rows later. Treating the pipeline as a single completion event hides the distinction between throughput and responsiveness.

ReFiles therefore considers the first realized row a more meaningful interactive milestone than the completion of enumeration. Item identity and basic display state can be published before expensive metadata. UI-thread work is issued in bounded batches so that the dispatcher can continue processing input and layout. Presentation keeps stable model identities when enrichment arrives instead of replacing every row. Virtualization limits the number of visual containers even when the logical result contains tens of thousands of items.

Progressive work also creates races. Navigation, refresh, filtering, or item replacement can invalidate an in-flight operation. Cancellation is useful but insufficient because a provider or platform handler may complete after cancellation was requested. The receiving layer must additionally verify that the result still belongs to the active browse generation and the same item identity before publishing it.

1.2.3 Ownership across asynchronous and platform boundaries

The file-manager domain contains many resources whose lifetime is not captured by ordinary managed references: streams, remote sessions, archive backends, cancellation registrations, Windows Shell objects, native buffers, preview handlers, and apartment-bound workers. A resource may be borrowed, shared, owned by one item, or tied to a broader provider session. Some resources require asynchronous cleanup, while others must be released on a particular thread or through a matching native allocator.

Implicit ownership makes failure paths especially dangerous. Capability creation can fail after earlier instances have already been created. A wrapper can fail while its inner object still needs disposal. Navigation can abandon work that later returns an owned result. Shutdown may need to aggregate cleanup failures without skipping remaining resources.

ReFiles addresses these cases by making ownership part of the architecture. Item-scoped capability instances are tracked and disposed in reverse construction order. Shared instances remain the responsibility of their composition root. Models support asynchronous disposal, and platform integration is kept behind schedulers and wrappers that preserve thread, allocator, and ownership requirements. Long-running storage operations may cross an out-of-process boundary rather than sharing UI state directly.

1.3 Design Position

The central design position of this thesis is that responsive, provider-agnostic browsing requires three boundaries to align:

1. **A semantic boundary:** generic code depends on item capabilities rather than provider identity.
2. **A publication boundary:** basic browse state is published progressively, while optional enrichment is scheduled and validated independently.
3. **An ownership boundary:** every resource has an explicit owner and cleanup path, including during cancellation, stale completion, partial construction, and application shutdown.

These boundaries reinforce one another. Lazy capabilities avoid constructing optional provider objects for every enumerated item. Progressive publication allows the UI to display identity before requesting enrichment. Explicit ownership lets delayed or rejected enrichment be cleaned up safely. Provider isolation, in turn, keeps platform-specific latency and failure from leaking into generic presentation semantics.

The architecture is divided into a UI-independent core, provider and platform implementations, presentation adapters, reusable controls, and an isolated operation host. The core owns storage references, stable identities, browse state, capability composition, session state, and operation intent. Presentation owns WinUI objects, dispatcher-bound publication, localized display state, viewport information, and conversion of encoded Core results into visual resources. This dependency direction allows the same storage and browsing semantics to be tested without constructing a WinUI visual tree.

1.4 Research Questions

The thesis is organized around the following research questions:

RQ1 How can optional item behavior from heterogeneous storage providers be composed without expanding a monolithic storage interface or introducing provider-specific branches into generic code?

RQ2 Which pipeline boundaries and measurements are needed to preserve interactive responsiveness while browsing and enriching directories containing tens of thousands of items?

RQ3 How can cancellation, stale-result rejection, and explicit ownership be combined to make asynchronous browsing and Windows platform integration predictable under failure and navigation races?

The questions connect structural and empirical concerns. RQ1 is evaluated primarily through the capability model and provider case studies. RQ2 motivates deterministic browse scenarios, architecture benchmarks, and measurements at the actual WinUI row-realization boundary. RQ3 is examined through lifetime contracts, fault-oriented tests, and scenarios in which work completes after its originating state has become stale.

1.5 Contributions

This thesis makes four practical contributions:

1. It presents a capability-oriented item model in which factories, deterministic combiners, wrappers, lazy resolution, caching, and lifetime ownership form one composition mechanism for heterogeneous providers.
2. It describes a progressive browse and presentation pipeline that separates resolution, enumeration, projection, bounded UI publication, visual realization, and optional enrichment.
3. It develops an explicit lifetime model for managed, asynchronous, remote, and Windows Shell resources, including cleanup after partial capability construction and rejection of stale asynchronous results.
4. It defines an evaluation method that distinguishes deterministic architecture overhead from machine-dependent Shell and filesystem behavior, with milestones including first Core batch, first presentation item, first realized row, dispatcher latency, notification count, model churn, and virtualization.

The implementation supports Windows filesystem and Shell items, archive browsing, and FTP/FTPS sources through the same core model. These providers serve as vertical case studies: they vary in latency, identity, stream ownership, metadata availability, threading requirements, and supported operations, yet reuse the generic browse, property, thumbnail, preview, and presentation paths where the corresponding capabilities are available.

1.6 Evaluation Strategy

The evaluation separates controlled architecture measurements from end-to-end platform scenarios. Isolated benchmarks measure cold and cached capability resolution, thumbnail-cache behavior, and browse enumeration/projection/publication at representative sizes. Deterministic WinUI scenarios exercise the production path from a synthetic provider through browse state, presentation adapters, view models, collection views, and an actually realized table row. Current scenarios cover 100, 1,000, 10,000, and 44,000 items, including progressive property enrichment.

The primary interactive metric is time to first realized row. Supporting metrics include time to the first Core batch, time to the first presentation item, enumeration completion, maximum and 95th-percentile dispatcher latency, stalls above several thresholds, collection notification count, unique view-model count, and maximum realized container count. Structural assertions verify correct final counts, bounded publication, stable model identity, and continued virtualization.

Real Windows folder scenarios are treated separately because they include disk and filesystem cache state, installed Shell extensions, property and thumbnail handlers, COM scheduling, and machine-specific behavior. Their results record environment metadata and are interpreted as scenarios rather than deterministic benchmark scores. This separation avoids presenting noisy hosted-runner timings as universal performance claims.

Comparative evaluation should emphasize architectural ablations before comparisons with other file managers. Useful baselines include progressive versus completion-gated publication, bounded batches versus one dispatcher callback per item, lazy versus eager capability creation, and cold versus cached capability resolution. These comparisons can isolate the effects of specific design choices using the same codebase and instrumentation.

1.7 Scope and Limitations

ReFiles is a Windows desktop application, and several case studies depend on Windows Shell and WinUI behavior. The architecture can express non-Windows providers, but this thesis does not claim platform-independent UI portability. FTP and archive support demonstrate provider diversity without representing every cloud, distributed, or content-addressed storage model.

The work also does not assume that architecture alone guarantees performance. Batch sizes, concurrency limits, caching policies, handler behavior, and UI layout remain empirical concerns. Timing measurements from real folders are sensitive to hardware, cache state, operating-system version, and installed extensions. The thesis therefore distinguishes stable structural invariants from environment-dependent measurements and avoids direct performance claims against other file managers unless equivalent instrumentation is available.

Finally, capability composition reduces provider coupling but does not eliminate semantic design work. Each capability still requires a clear contract for absence, cancellation, concurrency, result ownership, priority, and composition. The approach moves these decisions into explicit boundaries; it does not make them automatic.

1.8 Thesis Overview

The remainder of this thesis develops the capability model, progressive browsing pipeline, presentation boundary, and ownership rules in detail. It then describes the Windows, archive, and FTP implementations as case studies, evaluates the architecture through controlled and end-to-end measurements, and concludes with limitations and directions for future provider, search, operation, and observability work.

2 System Architecture

2.1 Design Constraints

ReFiles is shaped by constraints that are easy to hide in a small demonstration but unavoidable in a general desktop file manager. Storage is not uniformly local, paths are not universal identities, enumeration is not necessarily immediate, and item behavior cannot be inferred solely from whether an object is called a file or a folder. The Windows Shell namespace itself includes virtual objects in the same hierarchy as filesystem objects [2]. Archive entries and FTP objects introduce additional address and lifetime rules. Meanwhile, presentation must remain interactive while these backends perform work.

The architecture treats these as permanent conditions rather than exceptional cases. Five constraints guide the placement of behavior:

1. storage and application state must remain meaningful without a WinUI visual tree;

2. a provider must define identity and backend semantics without requiring generic consumers to recognize its concrete type;

3. browse results must be usable before enumeration and enrichment are complete;

4. asynchronous publication must remain correct after navigation, refresh, and disposal; and

5. native, remote, and managed resources must have explicit owners and cleanup paths.

These constraints lead to a dependency rule: higher layers adapt lower-layer state, but lower layers do not acquire presentation responsibilities. A useful test is whether a behavior still has meaning in a command-line or background host. If it does, the behavior belongs in the Core or below the Core boundary.

2.2 Layered Decomposition

The implementation is divided into four principal areas. `Files.Core` owns storage references, identities, models, provider composition, capabilities, browse sessions, selection, view settings, session state, and operation intent. The `Files` application owns WinUI views, view models, dispatcher adaptation, localization, visual selection, viewport reporting, and conversion of encoded data into UI resources. `Files.Controls` contains reusable controls without provider or application-session semantics. `Files.Operations` provides an out-of-process boundary for long-running or isolated file operations.

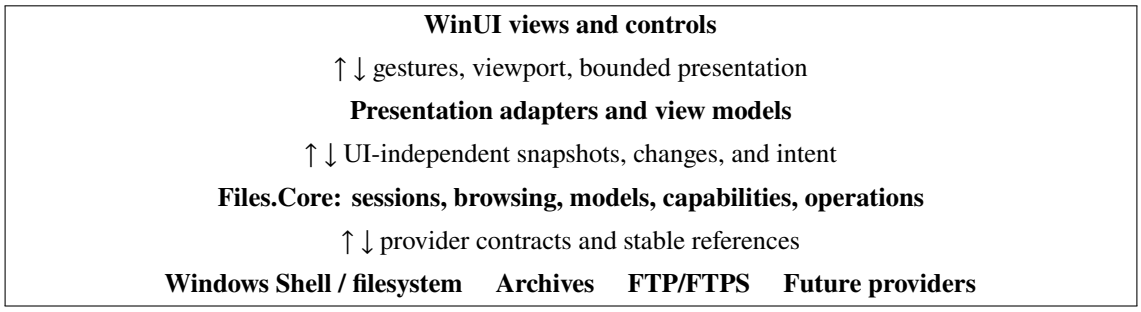

Figure 1: The principal dependency and adaptation boundaries in ReFiles. Arrows represent data and intent; compile-time dependencies continue downward from presentation toward Core contracts.

This decomposition is not a conventional three-layer split in which the middle layer merely forwards requests. The Core owns a substantial state machine. A `FilesCoreRuntime` exposes a storage workspace and a UI-independent application-session graph. The graph contains application, window, tab, pane, pane-content, and browse-session state. A non-visual host can enumerate workspace roots or invoke storage operations without creating a window, view model, or XAML object. The WinUI layer observes and adapts this graph instead of defining its storage semantics.

2.3 References, Identities, and Locations

Three concepts are kept distinct. A *reference* records enough provider-specific information to recover an item. An *identity* determines whether two observations refer to the same logical item for selection, reconciliation, and enrichment. A *browse location* describes a navigable destination and is resolved into a context that can enumerate items. Conflating these concepts with a textual path would exclude Shell virtual objects and produce unstable behavior for providers whose addressing rules differ from NTFS.

A `StorableReference` combines a source identifier, a provider item identifier, and an optional recovery address. A composite key gives generic dictionaries and projections a provider-qualified identity. Providers remain responsible for case sensitivity, rename behavior, server identifiers, virtual locators, and address normalization. Generic browse state consumes the resulting identity without interpreting it.

This separation matters during enrichment. A property result does not create a new item: it augments the presentation associated with the same key. Selection therefore survives property and thumbnail arrival. It also matters during failure recovery. An item may cease to exist while its reference remains serializable, and a provider may return an explicit not-found result rather than forcing Core to guess from a path.

2.4 Composition Root and Extension Points

One composition root constructs the provider registry, capability registry, location handlers, operation routes, caches, schedulers, and session roots. This makes the set of shared services and their disposal order visible. Provider addition is consequently a bounded task: implement source resolution and identity, supply a location handler for hierarchy, register optional capability contributors, and add operations supported by the backend. Generic view models and browse sessions do not receive a new provider branch.

The extension boundary is deliberately semantic. A thumbnail provider implements thumbnail production, not an FTP-specific flag. A folder-change implementation exposes normalized changes, not a watcher object to the UI. An archive provider exposes entries and streams through storage contracts while retaining responsibility for its decoder and shared source. This makes extensions independently testable and prevents platform details from becoming accidental application-wide conventions.

2.5 Architectural Invariants

Table 1 summarizes the invariants used during design and review. These are stronger than project organization rules because they constrain runtime behavior.

Table 1: Cross-cutting architectural invariants.

Invariant	Required behavior	Failure prevented
Core independence	Core state and provider contracts do not depend on WinUI objects.	Storage semantics leaking into visual lifetime or dispatcher rules.
Stable identity	Enrichment and sorting retain item identity.	Lost selection, duplicated rows, and stale updates attached to replacements.
Progressive work	Basic rows can publish before enumeration and metadata completion.	Completion-gated navigation and blank views.
Bounded publication	Collection changes and dispatcher work are coalesced into finite batches.	Notification storms and long UI-thread monopolization.
Current-state validation	Every delayed result is checked against its generation and identity.	Late work contaminating a newer location.
Explicit ownership	Every disposable result has a documented owner and cleanup route.	Leaks, double disposal, and apartment-invalid use.
Provider isolation	Generic code requests semantics rather than inspecting provider type.	Cross-cutting changes when adding a backend.

3 Capability-Oriented Item Behavior

3.1 Meaning of Capability

In ReFiles, a capability is an optional, typed semantic behavior available for one item. Examples include `IPropertySource`, `IThumbnailSource`, `IPreviewSource`, stream access, archive access, and folder-change observation. The term does not mean an operating-system permission or a security authority token. It describes what an item can correctly do through a particular contract.

This use resembles runtime interface discovery in COM, where `QueryInterface` determines whether an object supports an interface [3]. ReFiles differs in two important ways. First, the capability can be contributed by external factories rather than being implemented directly by the storage object. Second, multiple candidates may be composed or wrapped according to explicit policy. The capability set is therefore a semantic composition surface, not merely a cast.

The required storage model stays intentionally small. Absence is expected: an FTP item without a thumbnail source is valid, and an archive entry without rename support does not throw merely because generic code examined it. Consumers express dependency locally by requesting one typed contract and handling a missing result.

3.2 Registry and Resolution

A `CapabilityBuilder` registers three kinds of contributors for each capability type:

Factory Examines an item context and returns a candidate or `null`. Registrations include a priority, origin, and lifetime.

Combiner Selects or combines candidates deterministically when more than one implementation is available.

Wrapper Decorates the selected result with cross-cutting behavior such as caching, policy, or diagnostics.

The builder produces an immutable registry. Each item receives a lightweight `ICapabilities` object that holds its context and resolves types on demand. Resolution follows four stages:

1. add the core model itself when it directly implements the requested capability;
2. invoke registered factories and retain non-null candidates with their priority and lifetime;
3. apply the registered combiner, or accept the sole candidate when no combiner is required; and
4. pass the result through wrappers in registration order.

If several candidates exist without a combiner, resolution fails explicitly. Silent last-registration-wins behavior would make provider composition order an undocumented policy. A priority combiner is appropriate when one complete implementation should win; property, thumbnail, and preview subsystems may instead use specialized combiners that merge or fall back according to their own result semantics.

3.3 Lazy Resolution and Caching

Enumerating 44,000 items must not instantiate previews, property readers, thumbnail sources, archive adapters, and operation handlers for every row. ReFiles therefore resolves a capability only when a consumer requests its type. The resolved value—including a sentinel for absence—is cached per item. Repeated requests avoid factory execution, candidate allocation, and composition.

The cache is synchronized because presentation, prefetch, preview, and command-state paths may request behavior concurrently. Registry-side typed registration arrays are also cached so reflection-like casts are not repeated on each item. This design makes the cold path more expensive than a direct field access, while the common cached path becomes a type-keyed dictionary lookup. Section 7 separates these paths in the benchmark design.

Caching a missing result is important. Without a negative cache, scrolling across unsupported items could repeatedly execute the same factories. The capability set is static for the lifetime of an item model; if backend state changes so substantially that its capabilities differ, the provider creates or reconciles a new model through an explicit state transition rather than mutating the registry invisibly.

3.4 Ownership During Resolution

Factory composition is also a construction transaction. A factory may create an item-scoped disposable object; a later factory, combiner, or wrapper may then fail. The registry tracks every distinct item-owned instance as it is produced. On failure, already-created objects are disposed. If cleanup also fails, the construction and cleanup errors are combined so neither is lost.

Successful item-owned instances transfer to the item's capability set. Disposal occurs once, in reverse creation order. Direct capabilities supplied by the core model and registrations marked `Shared` remain owned by their existing model or composition root. Reference equality prevents the same object from being tracked twice when a combiner returns one of its inputs or a wrapper returns its inner object.

These rules make lifetime part of registration rather than a convention known only to one provider. They also support asynchronous disposal: when a capability implements `IAsyncDisposable`, cleanup awaits it before continuing. The .NET asynchronous dispose pattern exists specifically for resources whose cleanup itself is asynchronous [7]; remote sessions and operation hosts are natural examples.

3.5 Contract Design Checklist

A new capability is not complete when an interface has been added. Its contract must answer the questions in Table 2. This checklist prevents provider implementations from agreeing on a method signature while disagreeing on its meaning.

Table 2: Required decisions for a capability contract.

Decision	Question answered
Absence	Is unsupported behavior represented by a missing capability, an empty result, or a typed block reason?
Cancellation	At which points may work stop, and what resources must still be released?
Concurrency	May callers invoke the capability concurrently, or is access serialized by the provider?
Ownership	Are returned streams, sessions, buffers, or handles borrowed, owned, or tied to another lifetime?
Composition	Does priority choose one candidate, or are candidates merged, chained, or used as fallback?
Failure	Which failures are unsupported, missing, inaccessible, canceled, transient, or diagnostic errors?
Presentation	Which result remains UI independent, and where is it converted into a WinUI resource?

3.6 Answer to RQ1

RQ1 asks how heterogeneous optional behavior can be composed without a monolithic storage interface or provider branches in generic code. The ReFiles answer is a per-item, typed, lazy capability set fed by provider factories and governed by explicit combination, wrapping, and ownership rules. This does not remove backend differences; it assigns each difference to the smallest semantic contract that can represent it. The resulting extension cost is localized to provider composition and capability tests rather than distributed across browse and UI code.

4 Progressive Browsing and Presentation

4.1 Browsing as a Pipeline

C# asynchronous streams allow a producer to yield values over time and a consumer to process them with `await foreach` [6]. ReFiles uses this model to treat navigation as a sequence of observable milestones rather than one opaque task. The production path is:

Resolve location ⟶ enumerate items ⟶ project browse state
⟶ publish bounded changes ⟶ adapt to presentation
⟶ realize visible rows ⟶ enrich properties and thumbnails

Figure 2: The progressive browse and enrichment pipeline. Completion of a stage is not required before all useful work in the following stage can begin.

The distinction between first item, first Core batch, first presentation item, first realized row, enumeration completion, and enrichment completion is intentional. Only row realization proves that useful content crossed the layout boundary and reached the screen. A fast enumeration that causes a long dispatcher stall is not a responsive result.

4.2 Adaptive Bounded Enumeration

The current `BrowseSession` begins with a batch of 32 items, grows subsequent batches from 256 items, and caps a batch at 1,024 items. The exact values are implementation policy, not public API, but their shape encodes a trade-off. A small first publication reduces latency to first content. Larger later batches amortize projection, event, and dispatcher overhead. A maximum prevents throughput optimization from turning one update into unbounded work.

The batch boundary is preserved through the Core projection and then adapted to bounded presentation updates. One notification per item would make dispatcher and collection overhead proportional to the full item count with a large constant. One notification for an entire 44,000-item result would postpone all content and create a large synchronous mutation. Batching gives both the producer and presenter opportunities to yield.

Sorting and grouping complicate incremental publication. A naive implementation can rebuild or re-sort all N items for each arriving batch, turning progressive enumeration into repeated global work. ReFiles keeps an identity-indexed projection and debounces property-dependent sorting. Basic ordering can be updated while the dataset is incomplete; metadata-dependent order is reconsidered after relevant enrichment without replacing item identities.

4.3 Navigation Generations

Each navigation creates a monotonically increasing generation. A delayed publication is accepted only if it belongs to the current generation and the target item is still the same logical item. For a result r produced for generation g_r, the core condition is:

$$\text{publish}(r) \iff g_r = g_{\text{active}} \wedge \text{currentIdentity}(r.key).$$

Cancellation requests that obsolete work stop, while the generation check protects state when work does not observe cancellation in time. The two mechanisms solve different problems. A cancellation token reduces wasted work and releases cooperative providers early; current-state validation prevents corruption by an already completed or non-cooperative operation.

The rule applies to enumeration, property and thumbnail results, folder changes, delayed sorting, previews, and presentation callbacks. When navigation is replaced, the old context becomes stale before its resources are released. This ordering closes the interval in which a late result might otherwise pass a state check while cleanup is beginning.

4.4 Stable Projection and Selection

Browse state stores items by stable provider-qualified keys and exposes ordered snapshots. Selection is a set of identities, not ownership of visual row containers. This distinction allows a virtualized control to recycle a row while the corresponding item remains selected. It also prevents selection from disappearing when properties arrive or sorting moves an item.

Change notifications are reconciled through the same identity model. A create, update, move, or removal can be expressed as a bounded diff when the provider supplies sufficient information. A complete reload remains a fallback for ambiguous backend events, but it is not the default response to every notification. When changes race with enumeration, the generation and key checks decide whether a result can be applied.

4.5 Viewport-Driven Enrichment

Property, thumbnail, and preview work are not prerequisites for the first basic row. Presentation reports a viewport containing the first visible index, visible count, and look-ahead count. A `BrowsePrefetchCoordinator` creates separate property and thumbnail lanes, each with bounded parallelism. New viewport work cancels obsolete lane work and carries both a generation and a work identifier.

Visible and near-visible items receive priority because their enrichment affects the current interaction. An append-only enumeration does not invalidate useful prefetch for already-visible rows. Before publication, the coordinator checks the generation, the most recent work identifier, cancellation, and item identity. Best-effort prefetch failures do not invalidate the row; a foreground request may retry through the same capability.

This division assigns knowledge to the correct layer. Presentation knows which rows matter now. Core knows item identity and active browse state. Providers know how to obtain metadata. None of the three needs to assume the responsibilities of the others.

4.6 Presentation Boundary and Virtualization

Core publishes immutable, UI-independent presentation data and granular changes. The WinUI adapter converts those changes into dispatcher-bound collection mutations and view models. UI image creation and decode remain on the presentation side, while byte buffers, streams, and semantic thumbnail results remain below the boundary.

Virtualization is necessary but not sufficient. WinUI collection building blocks can realize only visible and near-visible elements when used with a virtualizing layout [8]. The data source must still avoid recreating every view model, issuing global command invalidation, or rebuilding grouping state for a small change. ReFiles therefore records unique view-model creation and maximum realized rows as separate structural metrics. A low container count cannot compensate for unbounded data-layer churn, and a stable data layer cannot compensate for a layout that realizes all rows.

4.7 Answer to RQ2

RQ2 asks which boundaries and measurements preserve responsiveness at large item counts. ReFiles separates resolution, enumeration, projection, presentation publication, row realization, and enrichment; it bounds work at each asynchronous-to-synchronous boundary; and it measures the milestones independently. The primary user-visible milestone is first realized row, while dispatcher latency, notification count, model churn, and realized-container count reveal whether the application remained interactive while background work continued.

5 Ownership, Cancellation, and Isolation

5.1 Ownership Hierarchy

The conceptual lifetime hierarchy follows application state: runtime owns application and workspace roots; application owns windows; a window owns tabs; a tab owns panes; a pane owns its current content; browse content owns a browse session; a browse generation owns its context and enumerated item models; an item owns its item-scoped capabilities. Shared caches, schedulers, and provider services remain owned by the runtime or provider composition root.

This hierarchy answers who initiates cleanup, but contracts also describe whether a returned value is borrowed, owned, or tied to another object. A view model generally borrows a Core model. A caller receiving an owned stream must dispose it. A remote data stream may retain the FTP control session needed to complete the transfer. A Shell object may remain valid only on its creating apartment. These distinctions cannot be inferred from garbage collection alone.

5.2 Cancellation Is Not Disposal

Cancellation is a request to stop; disposal releases ownership. A canceled provider may still hold a stream that must be closed, and disposing an object does not retroactively prevent a previously scheduled callback from running. ReFiles consequently applies the following replacement sequence:

1. mark the old generation stale by installing a new current-state boundary;
2. request cancellation of work owned by the old generation;
3. reject late results through generation and identity checks; and
4. asynchronously release resources when their contracts permit.

Cleanup paths are idempotent at the owning boundary. The capability set caches one disposal task so concurrent callers observe the same operation. It clears resolved values, then disposes owned instances in reverse order, continuing after individual failures and reporting an aggregate at the end. Construction failure uses the same principle: partial acquisition still creates an obligation to clean up.

5.3 Windows Shell and COM Apartments

The Windows provider must navigate both filesystem and virtual Shell objects. Shell COM objects can be bound to a single-threaded apartment. Microsoft documents that an apartment-model object lives on one thread and that interface pointers passed between apartments require marshaling [5]. ReFiles therefore treats the Shell as a constrained subsystem instead of replacing every blocking-looking call with arbitrary `Task.Run` work.

Message-pumped STA lanes separate metadata, extraction, operations, and preview workloads. Stable managed identities and locators cross generic async boundaries; COM objects are reacquired or used on an appropriate lane. Native pointer outputs retain their allocator contracts, and source-generated Win32 declarations preserve typed handles, HRESULT behavior, and safe-handle ownership. The rule is to keep unsafe or apartment-sensitive state at the platform boundary rather than allowing it into browse models.

Preview handling deserves a separate session boundary because third-party preview handlers may create child windows, require ordered callbacks, or fail independently of the item row. The Core exposes a hosted preview session contract; presentation supplies the UI host and lifetime. Navigating away disposes the session rather than retaining a handler because its file extension might match a future item.

5.4 Remote and Archive Resources

FTP and archives illustrate why a uniform `Stream` return type is not a complete ownership contract. An FTP data stream can depend on a control connection that must remain alive until transfer completion. An archive entry stream can depend on a decoder and a shared source stream. ReFiles returns owned abstractions that retain these dependencies and release them together. Credential resolvers remain separate from serialized identities so passwords do not enter recovery addresses, diagnostics, or persisted session state.

Archive browsing may use Windows Shell first and fall back to a SevenZip-based implementation. The fallback can require serial access to shared archive state or a password request. Both implementations expose the same Core semantics, but they do not pretend to share threading or cleanup mechanics. The provider boundary hides these mechanics while its capability contracts expose cancellation and ownership.

5.5 Operation Isolation

Copy, move, delete, and extraction can outlive a view, require progress and collision decisions, or fail after partial mutation. ReFiles represents operation intent and routes in Core, while `Files.Operations` provides an out-of-process execution boundary. The UI exchanges explicit requests, progress, and results rather than sharing view models or provider objects with the operation host.

Isolation reduces the damage from a long-running operation and creates a natural serialization boundary, but it does not replace transactional semantics. Providers must still define collision behavior, cancellation points, partial outcomes, and whether cross-source transfers require a coordinated copy-then-delete sequence. The important architectural gain is that these questions are expressed in operation contracts instead of being implicit in UI event handlers.

5.6 Answer to RQ3

RQ3 asks how cancellation, stale-result rejection, and ownership combine under races and platform constraints. The answer is a layered lifetime protocol: generation changes establish logical invalidity; cancellation reduces obsolete work; publication checks preserve state correctness; and disposal releases owned resources according to provider and platform rules. STA schedulers and process isolation add execution boundaries where a general task scheduler or shared object graph would violate those rules.

6 Provider Case Studies

6.1 Windows Filesystem and Shell

The Windows vertical slice resolves ordinary filesystem paths and Shell virtual items through one storage source. Filesystem items can use versioned identities and direct streams. Virtual items use encoded Shell identity and managed PIDL locators without exposing COM interfaces to generic code. Parent lookup, bounded folder enumeration, properties, thumbnails, change observation, preview handlers, and `IFileOperation`-based mutations are registered as capabilities and provider services.

The case demonstrates two benefits. First, an object can participate in generic browsing without mapping to a normal path. This follows the Shell namespace's own broader model of filesystem and virtual objects [2]. Second, capabilities keep rich Shell behavior optional. A file can expose typed properties and a preview handler, while another virtual object in the same browse result may expose neither. The row remains valid in both cases.

Windows also provides the strongest test of execution constraints. Properties, extraction, previews, and operations may call extensions with different cost and reliability. Separate STA lanes stop one category from becoming an undocumented global scheduler and make disposal ownership explicit at each session boundary.

6.2 Archives as Navigable Storage

Archives challenge the assumption that a folder is independently resolvable. Entries are often addressed within one container, and streams may share decoder state. ReFiles represents an archive as a browse location and its entries as storage models with stable container-qualified references. Shell archive folders are used when the platform supports them; a SevenZip-based path handles unsupported formats, encrypted archives, Windows 10 fallback cases, and remote input streams.

The archive source contributes hierarchy and readable content while omitting unsupported behavior. Preview can be reused from the stream capability: no archive-specific preview branch is needed. Likewise, an archive stored on FTP can flow through remote stream ownership into the same archive contracts. This composition is a practical example of why provider and capability axes are separate. "Archive" describes interpretation of content; "FTP" describes where the content is obtained.

6.3 FTP and FTPS

Each saved FTP connection becomes a source with a normalized, credential-free identity. Profiles cover plain FTP, explicit TLS, and implicit TLS, while credentials are supplied at runtime by an independent resolver. Listing results produce immutable files and folders with server-qualified identities and recovery addresses that contain no secrets.

FTP enumeration is naturally latency-sensitive. The provider performs one listing and yields models through the same asynchronous browse context as local sources. Listing metadata supplies properties without additional round-trips where possible. Owned data streams retain and complete their control sessions. Same-source create, rename, copy, move, and permanent delete are routed through storage-operation contracts. Remote thumbnails, polling change detection, SFTP, runtime profile registration, and cross-source transfer coordination remain explicit extension boundaries rather than partially implemented members.

6.4 Comparison

Table 3 compares the properties that most directly affect the architecture. The point is not that every provider has equivalent functionality, but that their differences are representable without changing the generic browse pipeline.

The case studies support RQ1 by showing that optional behavior composes along semantic axes, and RQ3 by showing that ownership remains provider-specific even when consumption is generic. They also support RQ2: all three feed the same progressive browse state, while provider latency is prevented from defining presentation policy.

Table 3: Provider case-study characteristics.

Concern	Windows filesystem / Shell	Archive	FTP / FTPS
Identity	Filesystem version or encoded virtual-item locator	Container plus entry identity	Source/server plus normalized remote identity
Enumeration	Bounded Shell/filesystem traversal	Decoder or Shell folder entries	Remote directory listing
Threading	COM STA and message-pump constraints	Shared decoder/source may serialize access	Asynchronous network sessions
Stream lifetime	File, Shell, or affine virtual stream	Retains archive and source dependencies	Retains data and control-session dependencies
Metadata	Shell properties and thumbnails	Entry metadata; reusable stream preview	Listing-backed properties
Operations	Shell `IFileOperation`	Format/backend dependent	Same-source remote operations
Notable absence	Capability varies for virtual objects	Many mutations may be unsupported	Native thumbnails and notifications not assumed

7 Evaluation

7.1 Evaluation Goals

The evaluation asks whether the implementation preserves the architectural shape claimed in the preceding chapters. It does not attempt a market comparison with other file managers. Equivalent end-to-end instrumentation is not available across products, and a faster number on one machine would not by itself demonstrate the capability or ownership model.

Evidence is divided into three levels:

1. deterministic Core benchmarks and synthetic scenarios isolate registry and browse-pipeline overhead;
2. contract and integration tests exercise disposal, stale-result rejection, provider behavior, and final state; and
3. deterministic and real-folder WinUI scenarios measure the production presentation path and actual row realization while recording environment-dependent context.

This separation follows the principle that a measurement should include only the variability needed to answer its question. BenchmarkDotNet provides jobs, diagnostics, and result exporters for controlled .NET microbenchmarks [11]. Shell handlers, filesystem caches, network servers, and UI layout require different scenario harnesses and interpretation.

7.2 Synthetic Core Browse Method

The repository contains a synthetic provider whose async iterator creates stable models without filesystem, Shell, network, thumbnail, or WinUI work. A `BrowseSession` navigates to the generated location at four sizes: 100, 1,000, 10,000, and 44,000 items. For each size, the scenario runs five times and selects the median run ordered by total completion time. It records time to the provider's first yielded item, time to the first Core `ItemsChanged` batch, total navigation time, process-wide allocated bytes over the operation, and notification count.

The measurements in Table 4 were collected on 28 August 2026 using Windows 11 Home build 10.0.26200, an Intel Core Ultra 7 155H with 22 logical processors, x64 project configuration, and .NET SDK 11.0.100-preview.4.26230.115. They describe this repository revision and machine. The platform was fixed to x64 so the benchmark and its source-generated Windows bindings used one consistent target architecture.

Table 4: Median-of-five synthetic Core browse results. Times are milliseconds.

Items	First item	First batch	Total	Allocated bytes	Notifications
100	0.0106	0.0804	0.9455	84,824	2
1,000	0.0220	0.1155	2.7912	833,896	4
10,000	0.0388	0.1530	47.6403	8,285,040	12
44,000	0.0452	0.1284	187.0732	35,991,216	46

7.3 Interpretation of Core Results

First-batch time remained below 0.16 ms in all four synthetic cases even though total completion increased with item count. This is the intended consequence of a small initial batch and async publication: the first Core notification is not gated on full enumeration. These numbers exclude storage and UI, so they should be read as pipeline overhead under an in-memory producer, not as a claim that users see a row in a fraction of a millisecond.

Notification count grew from 2 to 46 while item count grew from 100 to 44,000. The 44,000-item result therefore did not produce 44,000 item-level notifications. It is also not compressed into one completion notification. The observed counts support the intended bounded-batch shape: early content is published, later work is amortized, and the number of notifications grows far more slowly than the number of items.

Allocated bytes were approximately proportional to item count: about 848 bytes per item at 100 items and about 818 bytes per item at 44,000 items, using process-wide allocation differences from the scenario. This is not an object-size measurement and includes session, collection, task, and instrumentation allocations. It nevertheless shows no obvious superlinear allocation growth across these four points. A profiler and repeated BenchmarkDotNet allocation study would be required to attribute individual costs.

Total time is not strictly proportional across the small cases because sub-millisecond startup, scheduling, JIT, and garbage-collection effects are large relative to the workload. At 10,000 and 44,000 items, the measured medians were 47.6 ms and 187.1 ms. The evaluation does not infer a universal throughput rate from four points on one preview SDK. Its stronger conclusion is structural: first publication remained separated from completion, notification growth stayed bounded, and allocation growth was approximately linear in this controlled scenario.

7.4 Capability Benchmarks

The deterministic benchmark suite separately parameterizes capability resolution with 1, 4, and 16 registered factories. The cold operation creates an item capability set, resolves the requested type through factories and a priority combiner, and disposes the set. The cached operation requests the already-resolved type from the same set. A memory diagnoser records managed allocation. This construction isolates the central trade-off of the capability model: provider composition is paid on first semantic use, while repeated consumers use the per-item cache.

This evaluation does not report a capability timing table because a full statistically controlled BenchmarkDotNet dataset is required for a defensible latency estimate; a one-iteration smoke job proves execution, not performance. The benchmark remains reproducible in the repository, and future published revisions should record SDK, runtime, architecture, processor, job configuration, factory count, mean, error, standard deviation, and allocated bytes.

7.5 End-to-End Presentation Scenarios

The WinUI performance harness uses the production chain rather than a mock list control:

Synthetic provider → `BrowseSession` → presentation adapter → view model → collection view → details view → table row realization.

Scenarios cover details views at 100, 1,000, 10,000, and 44,000 items, plus 44,000 items with progressive property enrichment. They record first Core batch, first presentation item, first realized row, enumeration completion, maximum and 95th-percentile dispatcher latency, stall counts above 16, 50, and 100 ms, collection notifications, unique browse-item view models, and maximum realized rows.

The baseline uses structural failures rather than aggressive absolute timing gates on a noisy hosted runner. A wrong final count, first row realized only after completion, excessive realized rows, replacement view models for published identities, obviously unbounded notifications, or a catastrophic UI stall can fail the test. Absolute timings are emitted as machine-readable JSON for trend analysis. This makes a distinction between a regression that violates the architecture and natural variance that requires more samples.

Real-folder scenarios opt in separately and record Windows version, architecture, processor information, available memory, target folder, item count, cache-state hint, property and thumbnail settings, and environment notes. The first run may be cold or unknown; subsequent refreshes can be labeled warm. Results from different machines are not combined as if they were deterministic microbenchmarks.

7.6 Validity and Reproducibility

The synthetic provider gives strong control but weak ecological validity: it excludes storage latency, Shell extensions, decoding, network variance, and layout. The local results use one machine, one preview .NET SDK, and five repetitions selected by total median. Process-wide allocation can include runtime activity not caused directly by the session. Timing below a millisecond approaches scheduler and timer noise. These limitations are why the paper does not report the Core numbers as end-user latency.

The end-to-end harness improves ecological validity by including production presentation and row realization, but its environment is still synthetic. Real folders improve realism while reducing reproducibility. Installed preview and property handlers may block, cache state is imperfectly knowable, and machine power state affects UI latency. The three evaluation levels are complementary rather than interchangeable.

Reproducibility is supported by keeping scenario code, item counts, metrics, and result serialization in the repository. A stronger future evaluation should archive full benchmark JSON, exact commit identifiers, CPU power policy, runtime and Windows builds, raw per-iteration values, and ETW traces for anomalous stalls. Ablations of initial batch size, completion-gated publication, eager capability creation, and per-item notification would make the causal contribution of each design choice more explicit.

8 Related Systems and Design Context

8.1 Virtual Filesystem Abstractions

Desktop platforms have long placed heterogeneous resources behind file-like abstractions. The Windows Shell namespace presents filesystem and non-filesystem objects in one hierarchy [2]. ReFiles consumes this platform model but does not make it the universal Core abstraction: Shell identity, threading, and handlers remain one provider implementation.

GIO defines higher-level file, information, enumerator, stream, drive, volume, and mount abstractions, with network backends supplied through GVFS modules. Its design moves backends out of process to reduce dependency coupling and improve robustness, and it emphasizes asynchronous operations to avoid blocking a GUI main loop [9]. ReFiles shares the goals of higher-level storage contracts, cancellable asynchronous work, and isolated backend behavior. Its distinctive focus is the combination of per-item capability composition, browse-generation correctness, and a WinUI presentation boundary.

FUSE exposes a kernel-to-user-space protocol through which a daemon implements filesystem operations [10]. It is a powerful way to make a backend available to all filesystem consumers. ReFiles instead adapts providers inside an application-level semantic model. This avoids requiring every resource to implement a POSIX-like mounted filesystem and allows application-specific results such as previews, typed properties, and block reasons. The trade-off is that ReFiles interoperability is within its host rather than system-wide.

8.2 Interface Discovery and Composition

COM's `IUnknown` combines runtime interface discovery with reference-counted lifetime. Its identity rules require a stable `IUnknown` identity and a static set of interfaces for an object instance [4]. ReFiles borrows the useful idea that optional behavior should be requested by interface. It does not borrow COM reference counting or require the storage object to implement every interface. Factories can contribute behavior from the provider context, combiners can select or merge candidates, wrappers can add policy, and the item capability set owns disposable instances.

This distinction also separates the paper's "capability" terminology from object-capability security. ReFiles capabilities express optional service availability; they do not convey unforgeable authority. Security and credentials remain explicit provider and operation concerns. Calling the former `ItemFeature` would understate that consumers depend on a semantic contract rather than a UI feature flag; `Capability` captures the intended design more precisely.

8.3 Asynchrony, Cancellation, and Lifetime

.NET asynchronous streams provide progressive production and cancellation-aware consumption [6]. ReFiles builds a state protocol above the language mechanism. A token cannot prove that a result is current, so generation and identity checks remain mandatory. Likewise, `IAsyncDisposable` provides a cleanup mechanism [7], while the architecture must still decide who owns the resource, in which order cleanup occurs, and what happens when construction or cleanup fails.

COM apartment rules add a platform-specific constraint. Interface pointers cannot be passed freely across apartments, and single-threaded apartments associate objects with one thread [5]. Scheduler-mediated, message-pumped lanes are therefore correctness boundaries, not merely a performance optimization.

8.4 Incremental and Virtualized Presentation

Virtualized collection controls reduce visual realization to the current viewport and its cache region. WinUI's ItemsRepeater documentation describes virtualization as a property of the repeater together with a supporting layout [8]. ReFiles treats that as one component of a longer pipeline. Provider enumeration, Core projection, collection notifications, view-model identity, command invalidation, image decode, and layout must all be bounded. This is why the evaluation observes both first realized row and upstream structural metrics.

9 Discussion, Limitations, and Future Work

9.1 What the Architecture Achieves

The main result is alignment among semantic, publication, and ownership boundaries. Capabilities let a consumer ask for one optional behavior without learning provider identity. Lazy resolution prevents that flexibility from requiring eager construction for every enumerated row. Progressive browsing presents identity before expensive metadata. Generation checks prevent delayed capability results from crossing navigation boundaries. Explicit ownership ensures rejected or failed work still releases resources.

The provider case studies show that composition can occur across axes. FTP supplies remote bytes; archive support interprets those bytes as a hierarchy; stream preview interprets an entry's content; presentation converts the result into a visual. No single "FTP archive preview" branch is required. This is the strongest practical benefit of the model: extension becomes composition of independently testable semantics.

The design also makes omissions honest. A provider that cannot observe changes simply lacks the capability. A preview policy can return a typed block reason. Cross-source transfer remains a coordinator boundary instead of a method that some providers silently approximate. Explicit absence is preferable to a universal interface whose nominal support exceeds its semantic guarantees.

9.2 Costs and Failure Modes

Capability composition introduces indirection, type-keyed lookup, factories, and policy that a single-backend application might not need. Poorly designed capabilities can become a fragmented substitute for a monolithic interface. Too many fine-grained contracts may obscure which combinations form a usable workflow. Registrations can also encode hidden priority assumptions if origins and combiners are not observable.

Lazy resolution moves some failures from startup to first use. This is desirable for unsupported optional behavior but can make misconfiguration appear during interaction. ReFiles mitigates it with immutable registries, explicit failure when multiple candidates lack a combiner, origin diagnostics, and focused tests. A future registry validator could eagerly verify composition structure without constructing item-scoped implementations.

Progressive presentation creates intermediate states that sorting, grouping, selection, accessibility, and commands must tolerate. A simpler completion-gated design has fewer states, though worse time to useful content. ReFiles accepts the extra state-machine complexity because large and remote folders make completion an unsuitable interaction boundary. The cost must be controlled through immutable snapshots, keyed projection, and systematic generation tests.

9.3 Current Scope

The current provider set is evidence of heterogeneity, not coverage of the storage ecosystem. SFTP, cloud object stores, document libraries, content-addressed systems, offline synchronization, and conflict resolution remain future work. Search and tags have extension contracts but require a selected indexing backend. FTP lacks polling change observation and remote thumbnail policy. Archive mutation varies by backend. Cross-source transfers need explicit coordination and recovery semantics.

The UI is Windows-specific. Core independence makes non-WinUI hosts possible, but it does not prove portability to another desktop toolkit or operating system. Windows Shell integration remains a substantial part of the implementation, and third-party handlers can violate latency assumptions despite scheduler isolation.

The reported evaluation is intentionally limited. Only the synthetic Core scenario produced numeric results for this paper. Capability, UI, and real-folder harnesses are implemented, but a publication-quality multi-machine dataset and architectural ablation study remain to be collected. No comparison with Windows File Explorer, Dolphin, Nautilus, or another file manager is claimed.

9.4 Future Work

Four directions follow directly from the architecture:

1. **Provider breadth:** add SFTP and a cloud-backed source while preserving credential-free identity, progressive enumeration, and explicit stream ownership.
2. **Evaluation depth:** publish repeated BenchmarkDotNet, deterministic WinUI, and real-folder JSON; run batch-size and eager/lazy capability ablations; and archive environment metadata with each result.
3. **Observability:** correlate Core generation, provider work, dispatcher batches, row realization, and preview sessions in one trace so a user-visible stall can be assigned to the correct layer.

4. **Resilient operations:** formalize cross-source copy/move plans, resumability, partial outcomes, collision decisions, and operation-host recovery without sharing UI lifetime.

Additional work should examine accessibility during progressive enumeration. Announced set size, focus recovery, and selection must remain meaningful while the dataset grows and visual containers recycle. These concerns belong in the same structural evaluation as virtualization rather than being deferred to final visual polish.

10 Conclusion

This thesis presented ReFiles as a case study in building a responsive file manager across heterogeneous storage providers. The architecture does not force every item into one feature-complete interface. Instead, a small storage identity and hierarchy model is extended through typed capabilities supplied by factories, composed by explicit policy, wrapped with cross-cutting behavior, resolved lazily, cached per item, and disposed according to declared ownership.

The same discipline continues through browsing. Locations resolve into provider contexts that enumerate progressively. Core projects stable identities and publishes bounded batches. Presentation adapts those changes to WinUI and reports the viewport. Properties, thumbnails, and previews enrich relevant items later. Navigation generations, cancellation, and identity checks prevent obsolete results from entering current state.

Windows Shell, archives, and FTP/FTPS exercise different identity, threading, latency, and resource rules while reusing the generic pipeline. Synthetic Core measurements at up to 44,000 items showed first-batch publication remaining separate from completion and notification count remaining far below item count. The measurements are limited to the controlled pipeline and are complemented by an end-to-end methodology centered on actual row realization, dispatcher latency, model churn, and virtualization.

The broader lesson is that provider abstraction alone is insufficient. A responsive file manager requires its semantic extension model, progressive publication protocol, and lifetime system to agree. ReFiles makes those agreements explicit, testable, and extensible. That is the basis on which additional providers and richer operations can be added without turning each backend into a new application architecture.

References

[1] Files Community, "ReFiles source repository," 2026. [Online]. Available: `https://github.com/0x5bfa/ReFiles`. Accessed: Aug. 28, 2026.

[2] Microsoft, "Introduction to the Shell Namespace," Windows App Development Documentation. [Online]. Available: `https://learn.microsoft.com/en-us/windows/win32/shell/namespace-intro`. Accessed: Aug. 28, 2026.

[3] Microsoft, "COM Technical Overview," Windows App Development Documentation. [Online]. Available: `https://learn.microsoft.com/en-us/windows/win32/com/com-technical-overview`. Accessed: Aug. 28, 2026.

[4] Microsoft, "Rules for Implementing QueryInterface," Windows App Development Documentation. [Online]. Available: `https://learn.microsoft.com/en-us/windows/win32/com/rules-for-implementing-queryinterface`. Accessed: Aug. 28, 2026.

[5] Microsoft, "Single-Threaded Apartments," Windows App Development Documentation. [Online]. Available: `https://learn.microsoft.com/en-us/windows/win32/com/single-threaded-apartments`. Accessed: Aug. 28, 2026.

[6] Microsoft, "Generate and consume async streams," .NET Documentation. [Online]. Available: `https://learn.microsoft.com/en-us/dotnet/csharp/asynchronous-programming/generate-consume-asynchronous-stream`. Accessed: Aug. 28, 2026.

[7] Microsoft, "Implement a DisposeAsync method," .NET Documentation. [Online]. Available: `https://learn.microsoft.com/en-us/dotnet/standard/garbage-collection/implementing-disposeasync`. Accessed: Aug. 28, 2026.

[8] Microsoft, "ItemsRepeater," Windows App Development Documentation. [Online]. Available: `https://learn.microsoft.com/en-us/windows/apps/develop/ui/controls/items-repeater`. Accessed: Aug. 28, 2026.

[9] GNOME Project, "GIO 2.0 Overview," GLib Documentation. [Online]. Available: `https://docs.gtk.org/gio/overview.html`. Accessed: Aug. 28, 2026.

[10] Linux Kernel Developers, "FUSE," Linux Kernel Documentation. [Online]. Available: `https://www.kernel.org/doc/html/latest/filesystems/fuse.html`. Accessed: Aug. 28, 2026.

[11] BenchmarkDotNet Project, "BenchmarkDotNet Overview." [Online]. Available: `https://benchmarkdotnet.org/articles/overview.html`. Accessed: Aug. 28, 2026.